Agent OS: A Coordination Middleware for Large-Scale AI Agent Cluster

Nazmul Takbir (ntakbir), Waleed Alharbi (whalharb), Weiser Chen (yingchc6)

1. Introduction

The rapid advancements in large language models (LLMs) have led to a new era of multi-agent systems, where multiple LLMs collaborate to solve complex tasks that exceed the capabilities of any single model. By splitting a large task into smaller subtasks and steps, the overall performance can surpass that of using a single model alone. One body of research explores how tasks can be decomposed into subtasks to achieve optimal problem-solving capabilities. This line of work primarily addresses the NLP-side challenges of using multi-agent systems. However, several systems-level challenges also arise when multiple LLM calls are used to solve a single task, introducing complexities in resource allocation and utilization. In such scenarios, each user request is composed of a graph of LLM invocations, and a final response must be returned with minimal latency while optimally utilizing a distributed deployment of multiple LLM agents.

To address these challenges, we propose AgentOS, a coordination middleware for the distributed deployment of LLM agents. AgentOS automates the deployment and coordination of LLM agents to serve execution requests, where multiple execution requests within the same user task are interdependent. The AgentOS middleware improves the efficiency of the LLM agent cluster by optimizing scheduling, fault tolerance, checkpointing, load balancing, and task pruning.

2. Related Work

Solving complex tasks often requires more than a single LLM inference call. Instead, these tasks benefit from structured, multi-step reasoning in which multiple LLM invocations interact in a coordinated fashion. This paradigm, where intermediate steps or “thoughts” are generated, evaluated, and selected, naturally lends itself to a graph-structured execution model. One influential example is the **Tree of Thoughts (ToT) [8]** framework, which explicitly models the problem-solving process as a tree search over candidate thoughts. In the paper, the authors describe a challenging creative writing task where the model is tasked with generating a coherent four-paragraph passage that ends with four given random sentences. To solve this task, the ToT scheme first generates several high-level writing plans, each outlining how the passage could unfold. It then performs a voting step across these plans to select the most promising one, before proceeding to generate candidates for the output passage and finally voting again to select a single output. This process involves at least four sequential rounds of LLM calls, with each round comprising multiple LLM calls in parallel. Experimental results show that this structured approach yields significantly higher-quality outputs compared to relying on a single LLM invocation.

Building upon the structured multi-step reasoning of Tree of Thoughts (ToT), the **Graph of Thoughts (GoT) [9]** framework generalizes the idea further by allowing any thought to connect to any other through an edge in an explicit dependency graph. This removes the architectural constraint that every new idea must grow from a single parent; instead, GoT can branch, merge, loop back, or fan-in arbitrarily. On the other hand, **Decomposed Prompting [7]** introduces a framework where complex tasks are broken down into simpler sub-questions. A "decomposer prompt" generates a sequence of sub-queries instead of relying on a single prompt for all reasoning steps. Each sub-question is handled by a specialized sub-task module - a small prompt or an external tool (like a search engine or calculator). These modules are independent, allowing for reusability and separate improvement. A notable feature is its support for hierarchical and recursive reasoning; if a sub-task is too complex, it can be further decomposed.

MetaGPT [1] introduces a multi-agent framework inspired by the organizational structure of software engineering teams. Here, agents take on specialized roles such as product managers, architects, and developers, and interact according to predefined collaboration workflows. The framework effectively demonstrates how multiple specialized LLM agents can collectively generate non-trivial software applications. **AutoGen [2]**, developed by Microsoft

Research, presents a more flexible framework for enabling multi-agent conversations. It allows LLMs, humans, and tool interfaces to act as agents in conversational workflows. Each agent in AutoGen is programmable, with configurable behaviors, stopping conditions, and memory. Unlike MetaGPT, which leans on structured collaboration in predefined domains, AutoGen enables open-ended agent interactions that can evolve during execution. Its architecture supports diverse applications, including automated coding, reasoning, and peer review. AutoGen supports human-in-the-loop setups, where users can inject responses or corrections into the dialogue loop. This makes it highly suitable for real-world, semi-automated applications, though it lacks strong guarantees of long-term memory consistency or scalability in distributed deployments.

AIOS [3] shifts focus from task orchestration to infrastructure-level management by treating LLM agents as first-class OS-managed services, such as CPU or memory. It abstracts communication, scheduling, and lifecycle management through an agent OS layer, where LLMs function like computing cores. AIOS acts like a kernel, mediating agent interactions via a runtime with APIs for messaging and task routing. Its modular, decentralized architecture supports deployment across cloud, edge, or personal devices, enabling scalable, persistent multi-agent systems with migration and asynchronous communication. **Planet as a Brain [4]** extends AIOS into a geo-distributed system where agents run in "AgentSites" connected via publish-subscribe and inter-site protocols. AIOS acts as middleware within each site and helps delegate tasks across nodes, forming a mesh like a service mesh. A DHT with gossip protocol supports node discovery and membership updates. While conceptually ambitious, the system faces unresolved issues in routing, consistency, and trust.

AgentScope [5] offers a flexible platform for building multi-agent workflows with modular components like policies, memories, and tools. It emphasizes composability, fault isolation, and execution correctness. Unlike rigid scripting frameworks, it features a real-time execution engine that interprets plans, monitors progress, and handles failures. Its scope-based execution context gives each agent a sandboxed environment with local state and policies, improving debugging and reproducibility. With the combination of AutoGen's programmability with AIOS's robustness, AgentScope targets both research and production use cases. **Internet of Agents [6]** frames agent interaction as a networking problem, proposing that all agents, LLMs, humans, robots, be globally addressable and interoperable. It defines layered protocols like the OSI model, covering identity, messaging, and task delegation. Emphasizing heterogeneity, composability, and decentralization, it envisions an open, extensible agent web. Unlike task- or infrastructure-centric systems, it focuses on standardization and long-term evolution, similar to TCP/IP's role in global networking.

3. System Architecture

In this section, we introduce the service components in our system and the corresponding task workflow.

3.1 Service Components

Our system consists of five components: Gateway Server, Monitor Server, Database Server, Recovery Server, and Agent Proxy. In our failure model, only the Agent Proxy can fail and change address. The other components have fixed addresses and are not subject to any failure.

- **Gateway Server:** The gateway server handles the interactions between users and our system. The user submits a query and the task description to the gateway server. The server handles the following task creation and returns a task id as the response. The user then periodically polls the status of the task from the gateway server. More details in section 4.3.
- **Monitor Server:** The monitor server manages the view of memberships of the proxy clusters. The monitor only manages the membership of the proxies, not including other components because of our failure model. More details in section 4.4.
- **Database Server:** The database server stores the task entry on task creation and the subsequent progress updates. It also manages additional fields to deal with fault tolerance. More details in section 4.4.

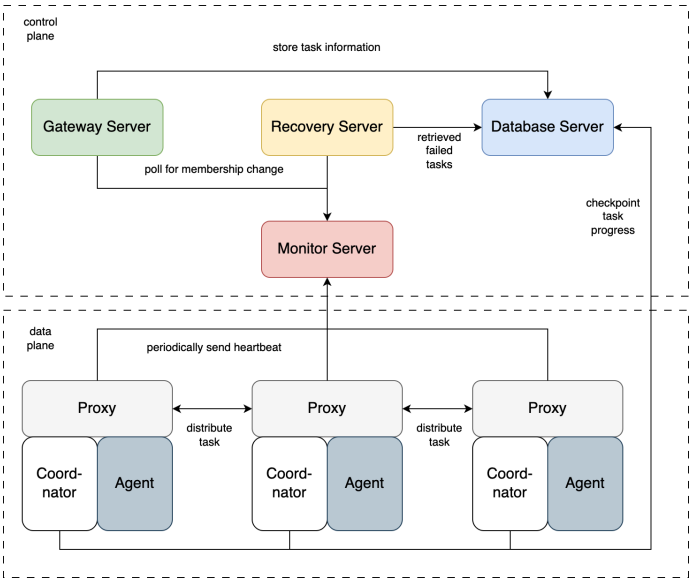


Figure 1. System Architecture

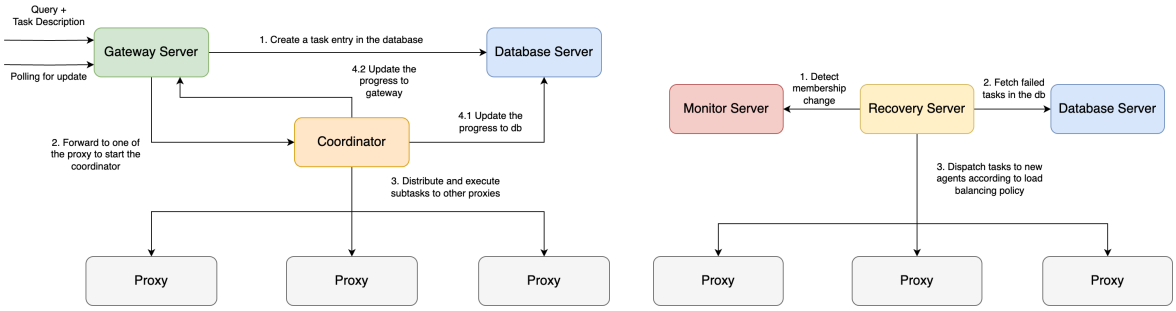


Figure 2. Normal Workflow (Left). Recovery Workflow (Right).

- **Recovery Server:** The recovery server recovers tasks on a failed agent proxy and allocates the failed task on other proxies in the system. More details in section 4.4.
- **Agent Proxy:** An agent proxy has two main roles: coordinators and agent. It also constantly reports its existence to the monitor server and gets the cluster membership back in the response. The reason behind the division of coordinators functionalities to every proxy is because of the better load balancing for failure recovery. If the coordinator is centralized into a server, it becomes a single point of failure. When distributing the coordinator, not only the coordinator can better utilize the resource on each proxy, parts of the tasks can still continue without being affected by the failed proxy.
    - **Coordinators:** The coordinator manages the execution of a request based on the client configuration parameters and the LLM techniques. We only support Tree of Thoughts as described in section 4.2. When an iteration ends, the coordinator performs the checkpointing process directly to the database server.
    - **Agent:**
        - A wrapper of LLM.
        - It handles the scheduling of the LLM requests. When a request from a coordinator arrives at a proxy, it will first be put into a queue. The request is then blocked until the task is scheduled and executed by the LLM. The scheduler will decide the next request to execute based on the scheduling policy. It also supports parallel execution using a semaphore.

3.2 System Architecture and Task Workflow

Figure 1 shows our system architecture. On the left side, the user sends the query and the task description to our system through the gateway server. The gateway server first creates a task entry in the database server and then forwards it to one of the agent proxies according to the load balancing policy. The gateway server keeps an entry to allow task updates from the coordinator. On receiving the forwarded request, a coordinator for that task will be created in memory to handle the task execution. In our current implementation, an execution is divided into several rounds. In each round, the coordinator passes through the generating phase and the voting phase. When the result is determined by the voter, the executor yields and creates a checkpoint to the database. In the meanwhile, the task progress is updated to the gateway as well as an optimization to save database queries from the gateway server. After the end of the task, the task is updated as completed and does the final update to the database and the gateway server as described before. The client polling ends when seeing the task status becomes completed and the gateway returns the result of the task to the client.

Figure 2 shows our task workflow. The left side is the normal workflow. The user sends the query and the task description to our system through the gateway server. The gateway server first creates a task entry in the database server and then forwards it to one of the agent proxies according to the load balancing policy. The gateway server keeps an entry to allow task updates from the coordinator. On receiving the forwarded request, a coordinator for that task will be created in memory to handle the task execution. In our current implementation, an execution is divided into several rounds. In each round, the coordinator passes through the generating phase and the voting phase. When the result is determined by the voter, the executor yields and creates a checkpoint to the database. In the meanwhile, the task progress is updated to the gateway as well as an optimization to save database queries from the gateway server. After the end of the task, the task is updated as completed and does the final update to the database and the gateway server as described before. The client polling ends when seeing the task status becomes completed and the gateway returns the result of the task to the client.

In the background, proxies continually send their heartbeats to the monitor server. The recovery process will start when the recovery server detected the membership change as seen in the right side of **Figure 2**. The recovery server fetches failed tasks in the database and starts a new coordinator for each task in other proxies according to the load balancing policy. More details in section 4.4.

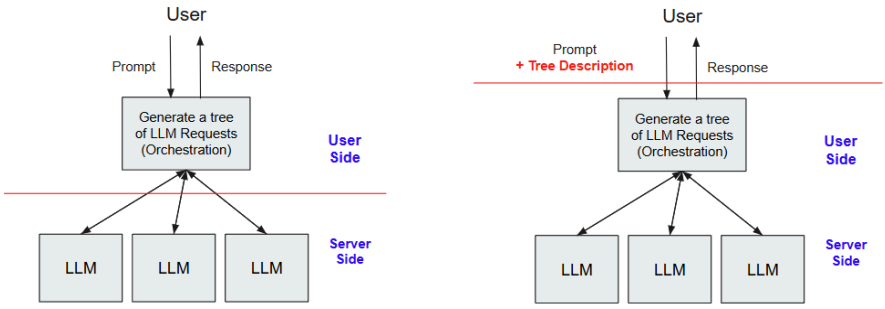
4. Implementation

4.1 Programming Stack

In our project, we design our API interface using RESTful API and Python as our main programming language. Services talk to each other via a set of predefined urls. **FastAPI [10]** is the HTTP framework we choose because of the workload characteristics. In our system, the bottleneck is the GPU resources and communication overhead. For the former one, both methods work fine. For the latter one, the frequent communication happens as each task may invoke 10x-20x LLM calls depending on the LLM techniques and configuration we use. If we choose thread as the basic concurrency primitive, the context switch would be huge due to the single-core nature of Python. Thus, the use of coroutine-based HTTP framework can achieve better performance in this case. For the testing, we use the Pytest framework and use a process to simulate each machine node. **The project code [11]** is put on GitHub.

4.2 Tree of Thoughts

In an ordinary Tree-of-Thoughts (ToT) workflow (left), a client-side orchestrator receives a prompt, spawns many parallel LLM calls across successive generation and voting rounds, and then delivers the final answer to the user. In AgentOS, we push almost all of this orchestration onto the server (right). The user now submits the prompt together with a description of the intended search tree, including depth, generation, and voting fan-outs. With full visibility into that graph, the coordinator can dispatch sibling node executions across all the agents. This server-side view enables smarter scheduling, load balancing, fault tolerance, and task pruning.



4.3 User-Gateway Interactions

The interaction between users and the gateway are asynchronous. The synchronization is more easily implemented but suffers from timeout and fault-tolerance. These two requirements are important in our workload as each task execution time spans at least several minutes. To prevent timing out from the client, we structure the interactions to be asynchronous by splitting the task creating and updating to two APIs. The task creation is fast as it simply stores an entry in the database and forwards the request to a proxy without waiting for the full execution. It returns the task id for task updating polling. To query the task status, the task id needs to be provided in the parameter. The gateway server returns waiting, completed, or aborted status. For waiting status, the client keeps polling until additional timeout is specified. When seeing the completed or aborted status, the task result is produced or canceled and the client stops and returns.

4.4 Member Management and Fault Tolerance

The monitor server manages the membership of proxies clusters. Each proxy server sends a heartbeat to the monitor server. The monitor server is assumed never to fail so the monitor address is hardcoded. The monitor and proxies agree on the same update intervals. Proxies send the heartbeat through random sleep to prevent abrupt change of the membership. The monitor waits for three update intervals to remove a dead proxy. The monitor doesn't care if the proxy is crashed or communicated has failed. The recovery process takes care of the conflicts and inconsistency.

The recovery server also constantly updates the membership of proxies from the monitor server. When detecting the membership change, the recovery process starts. The recovery server fetches all the tasks given the dropped member address from the database server with all the information required to recover the task, including task id, task progress, task description, and term (borrowed from Raft). The recovery process chooses the new agent based on the load balancing policy to distribute the task. The server retries if the new agent fails as well. When the retries run out, the task is marked as aborted. To prevent repeated recovery, all recovering agents are put in a recovery set and only be removed when all the tasks for an agent are recovered.. For the non-failing coordinator, it just ignores the failed request to the failed agent and proceeds using the successful invocation.

To prevent split-brain situations, a term field in the database is used to determine the task leader. The initial term for each task is 0. When an agent fails, the recovery process increments all the tasks controlled by the failed agents by 1. The new coordinator therefore contains the new term to proceed. When updating the progress to the database, the updates with terms smaller than the term field in the database will fail. The coordinator can only update the task if the term is greater than or equal to the database term for that task. When the old coordinator tries to update the database, it will fail and abort itself. This mechanism bound the task progress conflicts to at most one round.

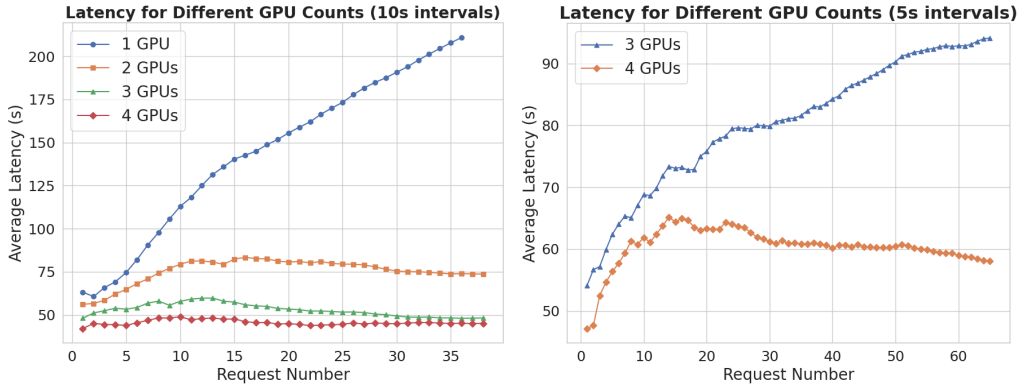
5. System Optimizations and Evaluations

In all experiments, our goal is to minimize the average latency of a user request: the time between receiving a user request, executing the entire tree of thoughts, and generating the complete final output. The workload is a

continuous stream of user requests, where the inter-arrival times are drawn from an exponential distribution with mean values of 5 seconds or 10 seconds. The graphs show how the running average latency of a request changes as more and more requests are served by the system. The part of the graph with a lower request count represents the transient state, whereas the part with a higher request count represents the stable state. In general, a lower running average latency in the stable phase of the graph is desirable.

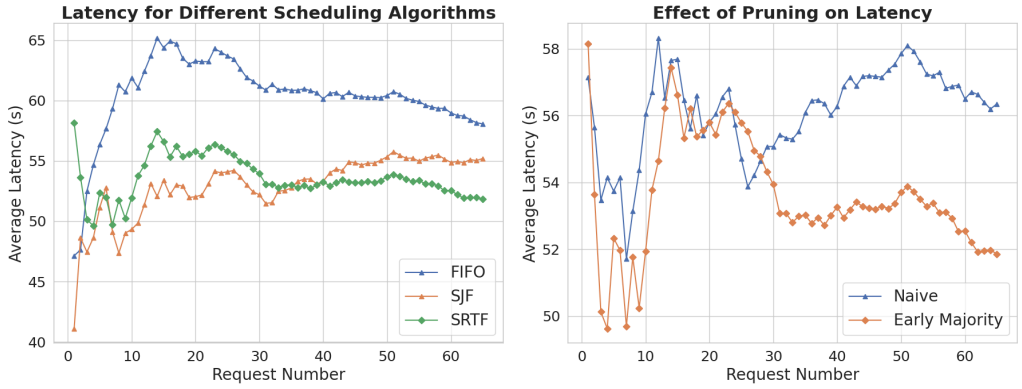
5.1 Horizontal Scaling

The results below show that AgentOS scales horizontally: as the number of GPUs is increased, the average latency decreases for both lower and higher rates of generating user requests.



5.2 Scheduling

The results below show the effect of different scheduling algorithms on the average latency. FIFO means tasks are prioritized according to the arrival time of their corresponding user request: those arriving earlier have higher priority. Shortest-Job-First (SJF) scheduling performs better than FIFO scheduling. Here, since we know the shape of the tree when the request arrives, we are able to crudely estimate the size of the user request in terms of the total number of LLM calls needed. Then, SJF scheduling prioritizes tasks that belong to requests with a smaller number of total LLM calls. Similarly, Shortest-Remaining-Time-First (SRTF) scheduling uses the number of remaining LLM calls needed to complete a request as a crude estimate of the time required to finish it. SRTF prioritizes requests with a lower amount of required time, and it performs even better than SJF.

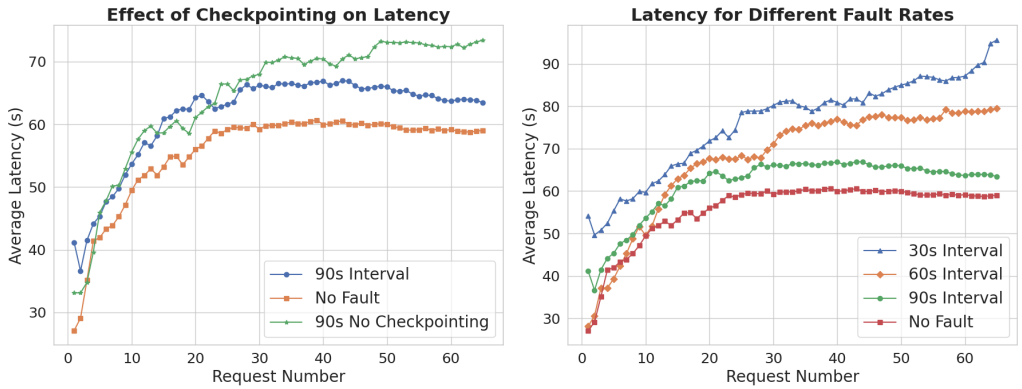


5.3 Pruning

In the Tree of Thoughts, there are two distinct phases: generation and voting. During voting, a group of LLM executions, say N, are created and tasked with determining the best output among a group of outputs. This means,

when a particular output has received ⌈N/2⌉ votes, it is guaranteed to be the winner, and we do not need to wait for the remaining LLM executions to finish. Thus, we can prune the tree and move to the next stage without waiting for the other voting executions to finish. The graph below shows that such an optimization significantly reduces the average latency.

5.4 Fault Tolerance and Checkpointing



When a group of LLM executions are used to generate the final output instead of a single LLM execution, it increases the likelihood of failure. A naive way to handle any fault would be to restart the entire execution from scratch. However, when the LLM executions are structurally related, we can employ a more efficient strategy. With the Tree of Thoughts approach, we implement checkpointing after every round or upon the completion of each level of the tree. Thus, if a failure occurs during the execution of Round 3, we can restart from Round 3 because the output of Round 2 has been checkpointed, rather than restarting from Round 1. The graph on the left illustrates the performance improvement achieved through this checkpointing mechanism. The graph on the right displays the effect of the failure rate on average latency. It demonstrates that while higher failure rates do increase average latency, the system consistently stabilizes to a certain average latency and does not become unbounded.

6. Conclusion and Future Work

In this work, we proposed Agent OS, a coordination middleware system for AI Agent clusters. The system is designed to support large-scale deployment and fault-tolerant execution. This system integrates components including the Gateway Server, Monitor Server, Database Server, Recovery Server, and Agent Proxy to support asynchronous task workflows, membership management, and recovery mechanisms. Agent proxies serve both as coordinators and LLM schedulers, by enabling distributed task execution and parallelism. The design avoids single points of failure by decentralizing coordinator roles. It also uses heartbeat-based monitoring and a term-based consistency model to manage recovery and update conflicts. These architectural choices allow Agent OS to support reliable multi-agent collaboration under dynamic conditions.

While AgentOS demonstrates the utility of designing a middleware that is aware of the tasks dependencies as well as the underlying distributed nature of the compute infrastructure to optimize resource usage, it is only just a step in the right direction. There is a lot of scope for future work:

- We built AgentOS primarily focusing on the Tree of Thoughts method of decomposing a request into a group of interrelated LLM executions. Ideally, we want to build AgentOS such that it is much more general: can handle graphs of any shape.
- In this work we performed fault tolerance at the granularity of rounds in the Tree of Thoughts structure. An even finer granularity of subtasks is likely to perform better.
- We benchmarked our system for a fault model where a server goes down temporarily and comes back up again soon. Ideally we should test other fault models which mix temporary and permanent faults.

- We also want to handle cases where the graph structure is determined dynamically.
- We also want to allow workflows that involve tool-calling and human-in-the-loop interactions.

7. References

[1] Hong, S. et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv preprint arXiv:2308.00352, 2024

[2] Wu, Q., et al. (2024, August). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In Proceedings of COLM 2024

[3] Mei, K., et al. (2024). AIOS: LLM agent operating system. arXiv preprint arXiv:2403.16971.

[4] Zhang, X. et al. Planet as a Brain: Towards Internet of AgentSites based on AIOS Server. arXiv preprint arXiv:2504.14411

[5] Gao, D. et al. AgentScope: A Flexible yet Robust Multi-Agent Platform. arXiv preprint arXiv:2402.14034, 2024.

[6] Chen, W. et al. Internet of Agents: Weaving a Web of Heterogeneous Agents for Collaborative Intelligence. In Proceedings of the 13th International Conference on Learning Representations (ICLR), 2025.

[7] Khot, T. et al. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. In Proceedings of the 11th International Conference on Learning Representations (ICLR), 2023.

[8] Yao, S. et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS), 2023.

[9] Besta, M. et al. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI), IAAI, and EAAI, 2024, Article 1972, pp. 1–9.

[10] FastAPI

[11] Project Repo